2026.10.23 · KO

GovBiz

한국어 검색에서 원문 근거까지, 서비스 경계와 배포를 함께 설계하다

HYBRID SEARCH / EVIDENCE RAG / ARCHITECTURE / DELIVERY

검색
Elasticsearch + Qdrant

검증
RRF + LLM + 원문

구조
Spring Boot + FastAPI

배포
Vercel + AWS

김건우 | Backend · AI Search · Architecture · Infrastructure

SOURCE PIN f9eca8d / 27 PAGES

문제에서 설계·검증까지 이어지는 기술 보고서

5인 팀 프로젝트에서 담당한 검색·RAG, 서비스·프론트엔드 구조, 배포·인프라를 중심으로 정리했습니다.

01	서비스와 애플리케이션 구조	데이터·계약 · MVVM과 Flux의 역할	03 - 13
02	검색·RAG와 근거 검증	조건 확인 · 벡터 · 순위 결합 · 인용 검증	14 - 22
03	평가와 결과 해석	한국어 검색 진단 · 개선 결과 · 한계	23
04	배포와 네트워크	검증 게이트 · 이미지 복구 · private routing	24 - 27

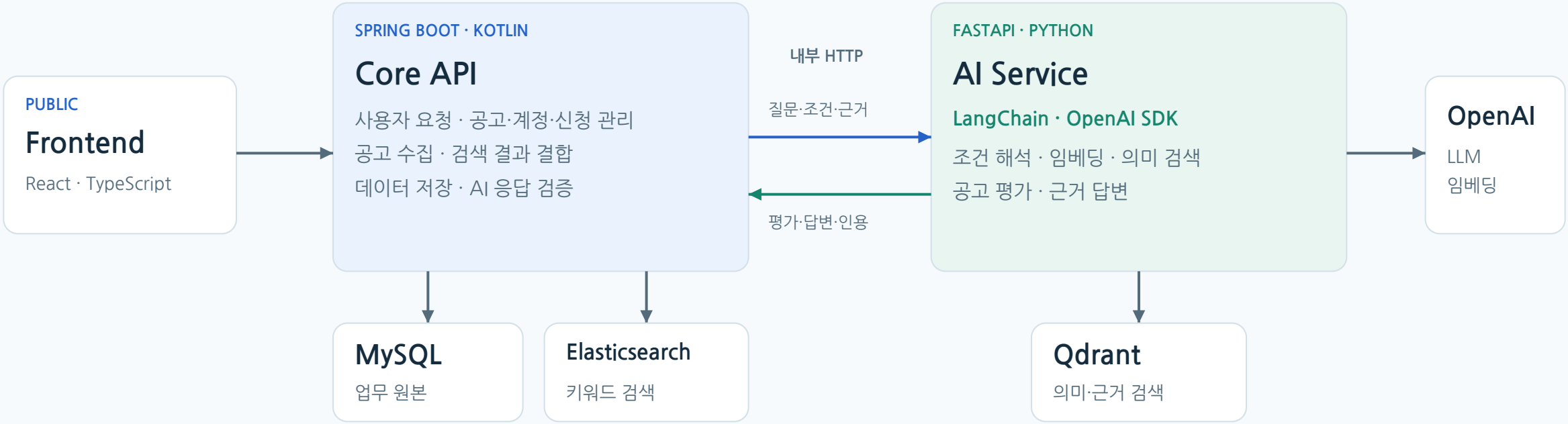
공고 탐색을 신청 준비·진행 관리까지 연결합니다

여러 공식 제공처의 공고를 찾고 근거를 확인한 뒤, 관심함을 중심으로 문서 작성·일정·협업을 이어가는 서비스입니다.



Python 라이브러리를 활용하고, 업무와 AI 실행을 나눴습니다

Python의 LangChain·OpenAI SDK를 활용하고, 업무 데이터 처리와 AI 실행의 책임을 분리했습니다.



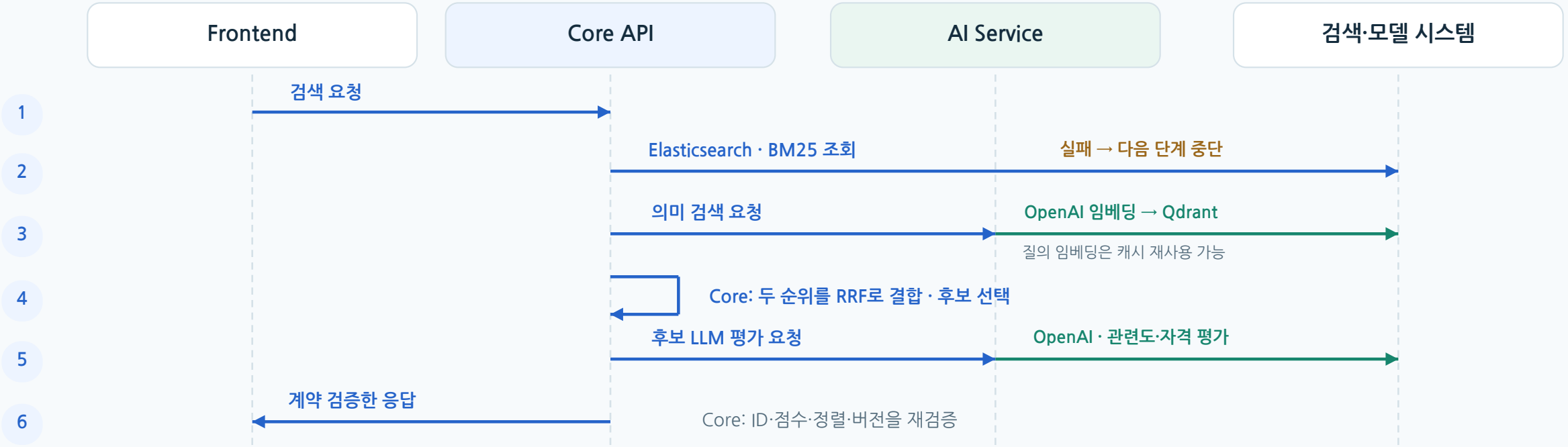
원본·검색 색인·임시 결과의 역할을 나눕니다

공고 원본과 업무 기록은 Core API가 관리합니다. AI Service는 MySQL에 직접 접근하지 않고 전달받은 데이터로 검색·생성을 실행합니다.



검색 호출 순서로 비용과 장애 경계를 통제합니다

Core가 검색 순서를 조율합니다. Elasticsearch 성공 뒤에 의미 검색을 호출하고, 후보를 합친 뒤 LLM으로 평가합니다.



HTTP 성공 뒤에도 결과를 검증하는 경계를 둡니다

공개 DTO, AI 통신 DTO, 내부 실행 결과를 분리하고 생산자·소비자 양쪽에서 요청과 응답의 계약을 확인합니다.

CLIENT
공개 요청
POST /api/v1/
support-programs/search

→

CORE
내부 요청
후보 ID · 질의
scoringVersion

→

AI
생성 응답
Pydantic + Service
모델 결과 검증

→

CORE
채택·변환
집합 · 점수 · 순서
중복 · 버전 재검증

오류를 정상 검색 결과와 구분

502 잘못된 AI 응답·상위 오류

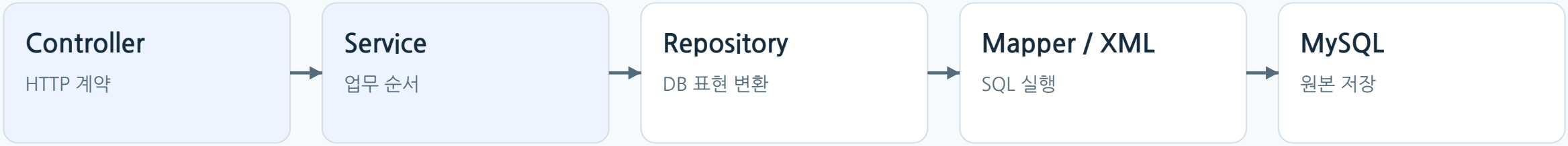
503 AI Service 사용 불가

504 AI 통신 시간 초과

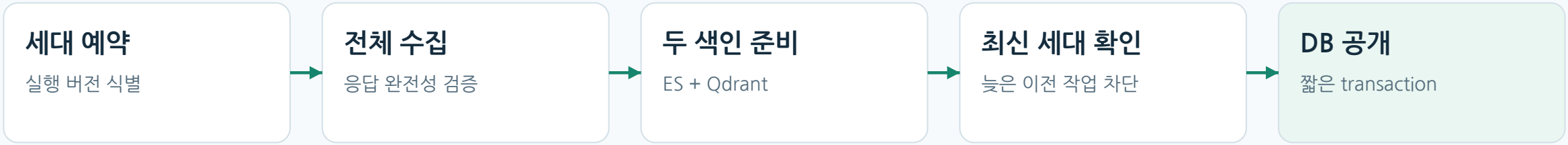
계층은 책임을 나누고, 공개는 짧은 트랜잭션으로 묶습니다

기능별로 HTTP·업무·외부 통신·영속성의 책임을 나눕니다. 외부 수집과 색인은 DB 트랜잭션 밖에서 수행합니다.

실행 경로



동기화 공개 경계



검색 준비를 마친 최신 수집 결과만 반영합니다

검색 데이터를 모두 준비한 뒤, 같은 제공처에서 가장 최근에 시작한 작업인지 확인합니다.



검색 결과 확인 현재 MySQL에서 제공하는 공고와 내용이 같은 검색 결과만 사용합니다.

검색 데이터 준비 순서

Elasticsearch 준비 후 Qdrant를 준비합니다.
40건이면 16·16·8건으로 요청하며, 공고마다 벡터를 따로 준비합니다.

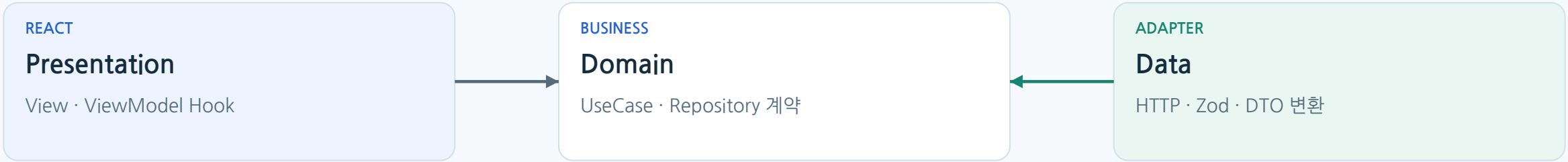
작업 번호 비교 예시

1번 작업 뒤에 2번이 시작됐다면 1번 결과는 반영하지 않습니다.
2번이 실패해도 마지막으로 정상 반영한 공고를 유지합니다.

화면 책임과 상태 변경 방향을 따로 설계했습니다

주요 업무 기능에 Clean Architecture의 의존 방향, Custom Hook 기반 MVVM, Redux Toolkit의 Flux 흐름을 적용했습니다.

코드 의존 방향



다음 두 장에서 분리해 설명하는 책임

MVVM / Hook ViewModel

View와 조회·화면 로직·업무 호출의 책임 분리

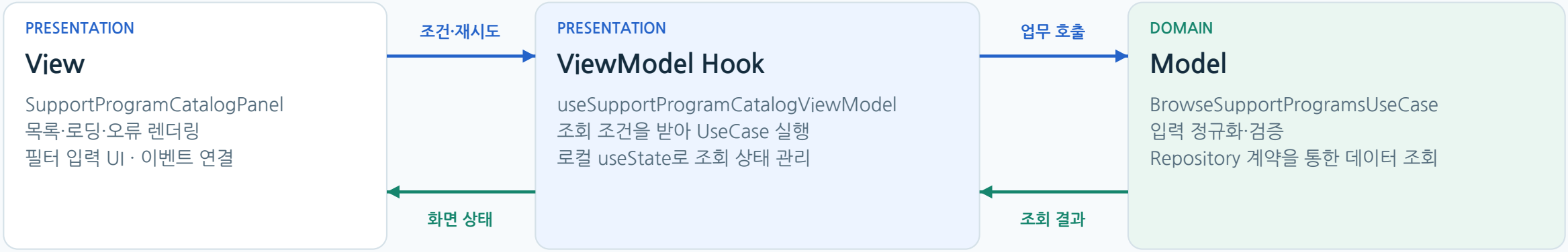
Flux / Redux

ViewModel 뒤에서 공유 상태의 갱신 방향 관리

MVVM은 역할 분리, Flux는 상태 갱신 방향을 설명합니다. 채팅의 ViewModel도 Hook으로 구현하며 Redux와 함께 사용합니다.

MVVM: Hook ViewModel로 화면과 업무 호출을 분리합니다

공고 목록 화면의 사례입니다. ViewModel이 조회 결과·로딩·오류·재시도를 제공하고, 업무 규칙은 Model 영역의 UseCase가 처리합니다.



View에 제공하는 계약

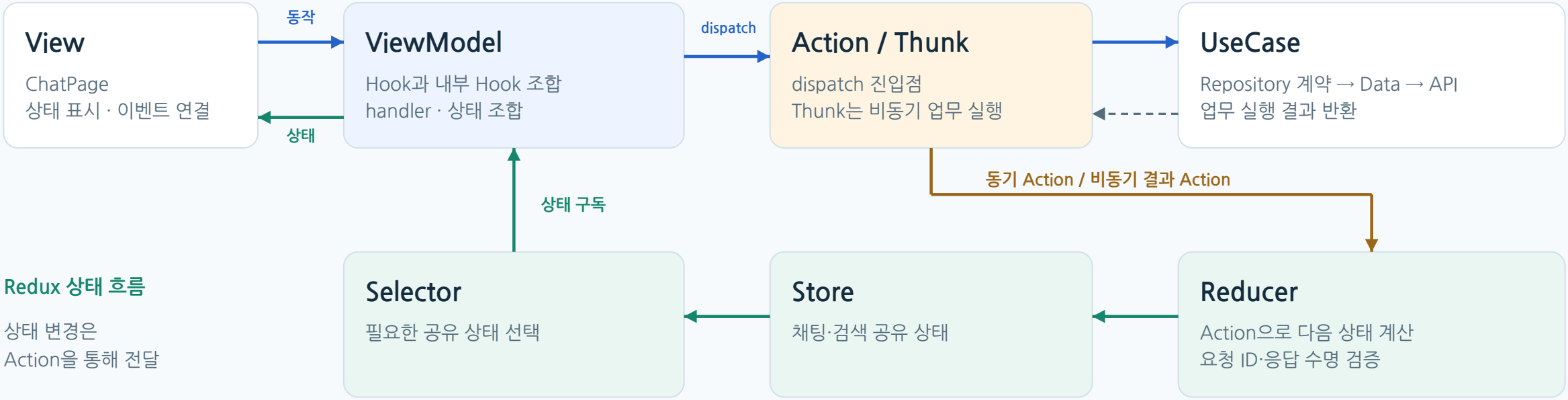
phase · data · 필터 옵션 · retry

ViewModel의 경계

View는 UseCase·Repository를 직접 실행하지 않습니다.

Flux: Redux의 상태 흐름도 ViewModel을 통해 연결합니다

채팅 화면의 사례입니다. ViewModel이 사용자 동작과 화면 상태를 중계하고, Redux는 Action → Reducer → Store의 단방향 갱신을 담당합니다.



공통점: ViewModel이 View와 업무 로직을 분리 · 차이점: 조회 로컬 상태와 Redux 공유 상태의 관리 방식

이전 요청의 늦은 응답이 최신 화면을 덮지 않게 합니다

공유 상태, 화면 로컬 상태, 진행 요청의 수명을 분리하고 requestId로 응답을 현재 작업과 대조합니다.

응답 순서가 바뀌는 상황



STATE OWNERSHIP

보존할 상태로 구분

Redux: 채팅·검색 상태
Hook: 페이지 전용 상태·DOM
요청 레지스트리: AbortController·타이머

브라우저 요청을 취소해도 서버에서 이미 시작한 OpenAI 실행까지 중단되는 것은 아닙니다.

AI의 조건 제안은 사용자 확인 후 검색에 반영합니다

후속 대화의 해석과 검색 실행을 분리합니다. 확정 조건과 미확정 제안을 따로 보관하고, 사용자가 확인한 조건으로만 검색합니다.

01 / INPUT
사용자 질문
"부산으로 변경"
기존 조건과 함께 전달

→

02 / AI
변경 목록 해석
REGION = 부산
현재 발화의 인용 근거

→

03 / CORE
조건 제안·검증
변경한 필드만 제안
나머지 조건은 유지

→

04 / USER
확인 후 검색
이 조건으로 검색
확정 조건·검색 스냅샷 저장

바로 검색하지 않는 두 가지 응답

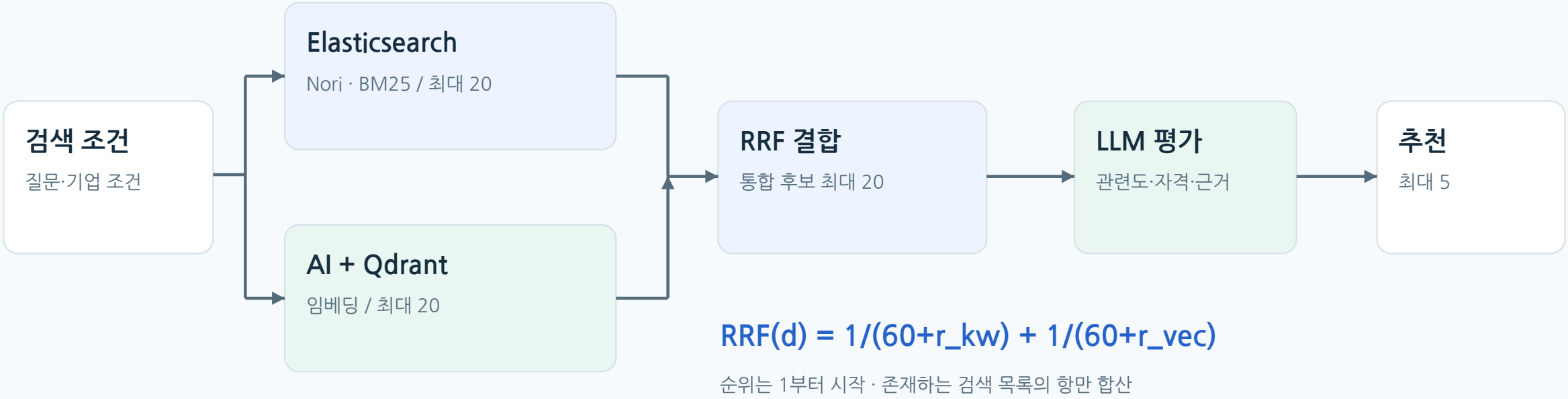
추가 확인 필요	정확한 조건을 질문하고 초안을 유지
설명 답변	검색·조건 변경 없이 결과를 설명

검색 결과는 서로 다른 출처를 결합

공고 정보·신청 기간	Core MySQL
추천 점수·이유·자격 검토	AI 평가 + Core 검증

키워드와 의미 후보를 RRF 순위로 결합합니다

서로 다른 검색 점수의 척도를 직접 더하지 않고, 두 목록의 순위를 동일 가중치로 통합합니다.



$$RRF(d) = 1/(60+r_{kw}) + 1/(60+r_{vec})$$

순위는 1부터 시작 · 존재하는 검색 목록의 항만 합산

실제 호출은 Elasticsearch → AI 의미 검색 순서입니다. 키워드 검색 장애를 유료 질의 임베딩 전에 확인합니다.

공고를 찾는 벡터와 답변 근거 벡터를 분리합니다

Qdrant를 공유해도 검색 목적과 단위가 다릅니다. 공고 검색과 공식 원문 근거 검색은 서로 다른 컬렉션에서 실행합니다.

DISCOVERY
공고 단위 검색
공식 API 4개 제공처
관련 공고를 찾는 단계

→

PREPARE
Core 검색 문서
제목·기관·대상·지역
신청 기간·지원 내용

→

EMBED
AI 임베딩
검색 문서 1건마다
벡터 1개

→

QDRANT
공고 컬렉션
공고 ID + 내용 해시
제공처 코드 + 벡터

EVIDENCE
원문 근거 검색
기업마당 상세 HTML
답변의 근거를 찾는 단계

→

PREPARE
Core 정제·청킹
최대 1,500자 / 청크
문서당 최대 50개

→

EMBED
AI 임베딩
원문 청크 1개마다
벡터 1개

→

QDRANT
근거 컬렉션
문서·청크 ID / 해시·순서
청크 원문 + 벡터

같은 문서 버전·임베딩 설정의 벡터는 재사용하고 누락분만 생성합니다. 질문 벡터는 검색에만 사용합니다.

두 검색에 함께 오른 후보가 어떻게 합쳐지는지 봅니다

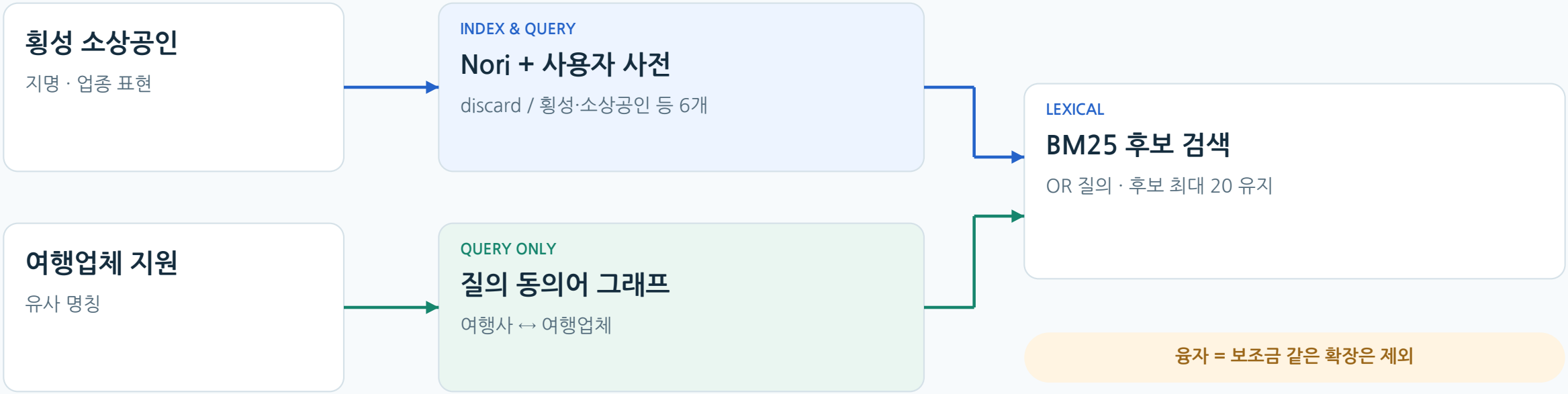
설명용 순위를 직접 계산한 예시입니다. 실제 공고의 검색 결과나 성능 측정치는 아닙니다.



형태소 분석과 유사 표현의 누락을 단계별로 좁힙니다

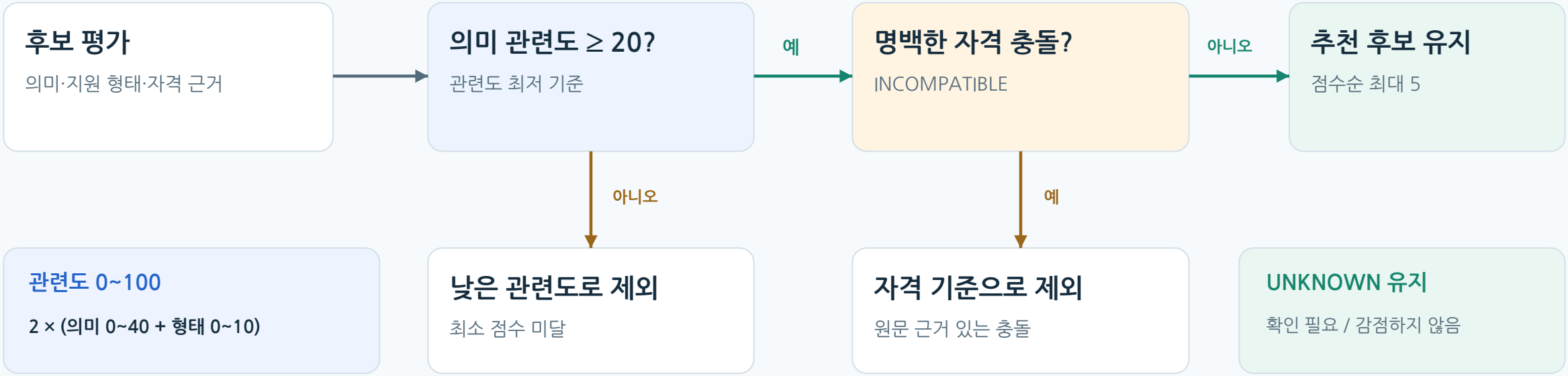
분해 모드·사용자 사전·질의 동의어를 보완하고, 금융 방식이나 신청 범위를 바꾸는 확장은 제외했습니다.

입력의 표현 차이



관련도 기준과 자격 기준을 서로 다른 분기로 적용합니다

자격을 확인하지 못한 후보는 관련도에서 감점하지 않습니다. 그림은 선별 규칙의 요약이며 호출 순서도는 아닙니다.



관련도 100점은 선정 확률이 아닙니다. 자격 정보가 부족하면 UNKNOWN으로 유지해 추가 확인이 필요함을 표시합니다.

현재 원문의 청크만 검색하고 인용을 다시 검증합니다

추천 후보 검색과 별도로, 선택한 기업마당 공식 HTML의 근거 청크를 검색해 상세 질문에 답합니다.

공식 HTML
Core 수집·저장

→

결정적 청킹
최대 50 / 각 1,500자

→

Qdrant 검색
허용 청크 중 최대 5

→

원문 복원
Core의 현재 청크

→

LLM 답변
근거 내 작성

검색 결과 검증

공고·청크 ID / 내용 해시 / 문서 ID / 순서
중복 / 유한한 점수 / 결과 정렬

답변·인용 계약

ANSWERED → 인용 필수
근거 부족 → INSUFFICIENT_EVIDENCE

검증 대상은 기업마당 HTML의 출처·구간·인용입니다. 이 검증만으로 답변 전체의 의미적 정확성을 보장하지는 않습니다.

검색 결과와 최종 인용을 서로 다른 시점에 검증합니다

Core는 검색 결과를 먼저 원문과 대조합니다. 검증한 청크로 AI 답변을 만든 뒤, 인용 계약을 다시 확인하고 원문 발췌를 조립합니다.



[S17] 원문·인용 검증 [S18] 인용 번호 복원

긴 식별자 복사 대신 짧은 번호를 선택하게 했습니다

실측에서 64자리 청크 ID가 63자리로 생성된 실패를 확인하고 모델과 코드의 역할을 나눴습니다.



목표 공고 Hit@20은 295건에서 299건으로 늘었습니다

고정 공고 1,422건과 개발 중 사용한 AI 작성 질문 300개에서 Nori v1·v2를 비교한 로컬 ES 실험 기록입니다.

목표 공고 적중 수 / 300



MRR@20 / 목표 공고 순위의 역수 평균

0.906111 → 0.933764

최종 추천 순위가 아닌 키워드 후보 순위

누락 5개 중 4개 회복

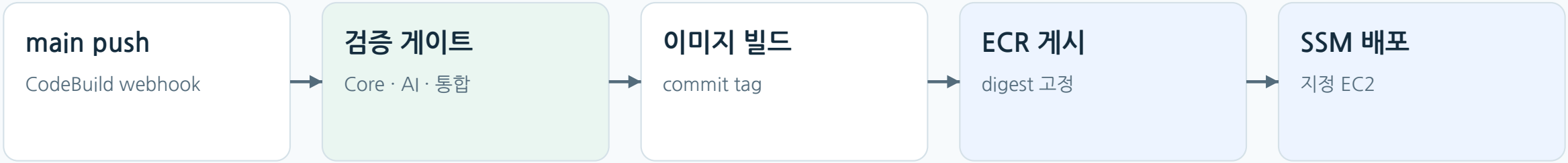
기존 적중 목표의 top-20 이탈 0건

순위 하락 질문 9개 · 잔여 누락 1개

AI 작성 질문을 사용한 키워드 후보 검색 실험입니다. 사람 검증 정답셋·전체 Recall·최종 추천 정확도·운영 성능과 구분합니다.

검증을 통과한 이미지만 ECR과 운영 환경으로 보냅니다

Frontend는 Vercel Git 연동으로 배포하고, Backend는 CodeBuild webhook으로 시작합니다. CodeConnections는 소스 연결을 담당합니다.



CodeBuild 검증 범위

Core: JDK 21 clean build·테스트
AI: 잠금 파일·전체 pytest·패키지 빌드
통합: Qdrant·Compose·Nginx와 평가 도구

버전 확인

커밋 태그 + immutable digest
게시 전후 최신 main 확인
오래된 대기 빌드 반영 차단

통합 검증에서는 OpenAI 스텁을 사용합니다. 배포 전 테스트 통과와 실제 LLM 답변 품질은 별도로 검증합니다.

상태를 확인하며 교체하고 이전 이미지 복구를 시도합니다

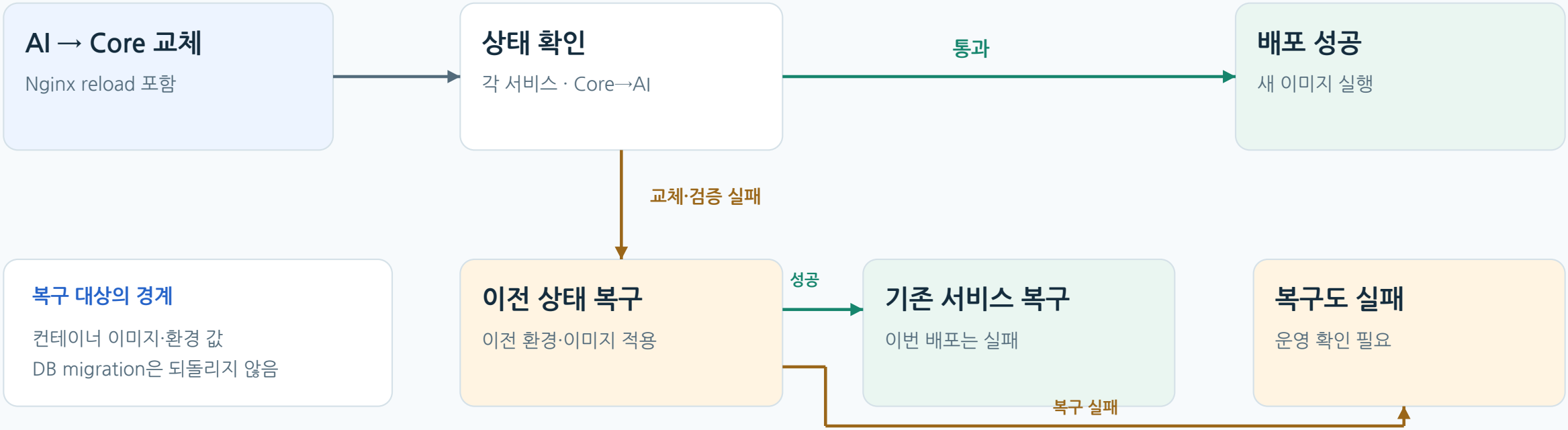
호스트 배포 잠금과 사전 점검 뒤 AI를 먼저 교체하고, Core와 프록시 연결을 확인합니다.



단일 EC2의 컨테이너 교체 중 서비스가 중단될 수 있습니다. 이미지 복구는 이미 적용된 DB migration을 되돌리지 않습니다.

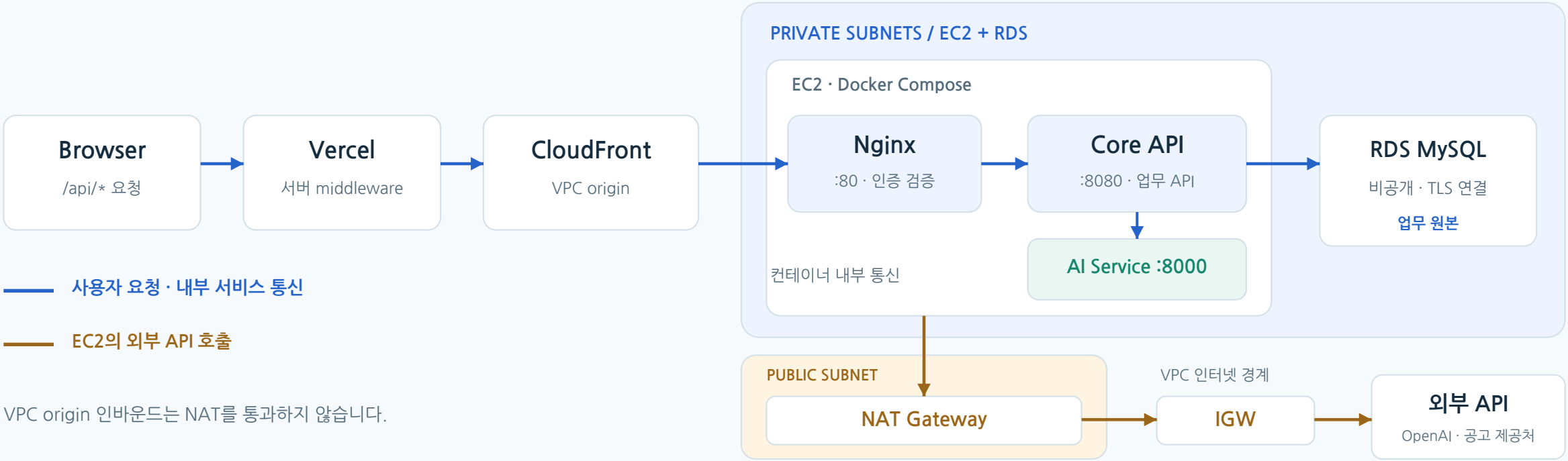
교체 실패와 복구 실패를 별도 결과로 남깁니다

배포는 상태 확인을 거쳐 종료합니다. 이전 이미지 복구에 성공해도 이번 배포가 성공한 것으로 바꾸지 않습니다.



인바운드는 VPC origin, 아웃바운드는 NAT로 분리합니다

EC2와 RDS는 프라이빗 서브넷에 둡니다. 사용자 API 요청은 Vercel과 CloudFront를 거치고, 서버의 외부 호출은 NAT Gateway를 사용합니다.



VPC origin 인바운드는 NAT를 통과하지 않습니다.